一、Pycharm 灵码进阶使用

1.1 智能问答

作为您的智能编码助手，当您进行编码工作遇到疑问时可以使用智能问答模式，帮您答疑解惑，快速生成建议代码，生成代码注释、代码优化、解释代码、修复代码问题、排查编译报错等。

1.1.1 操作步骤：

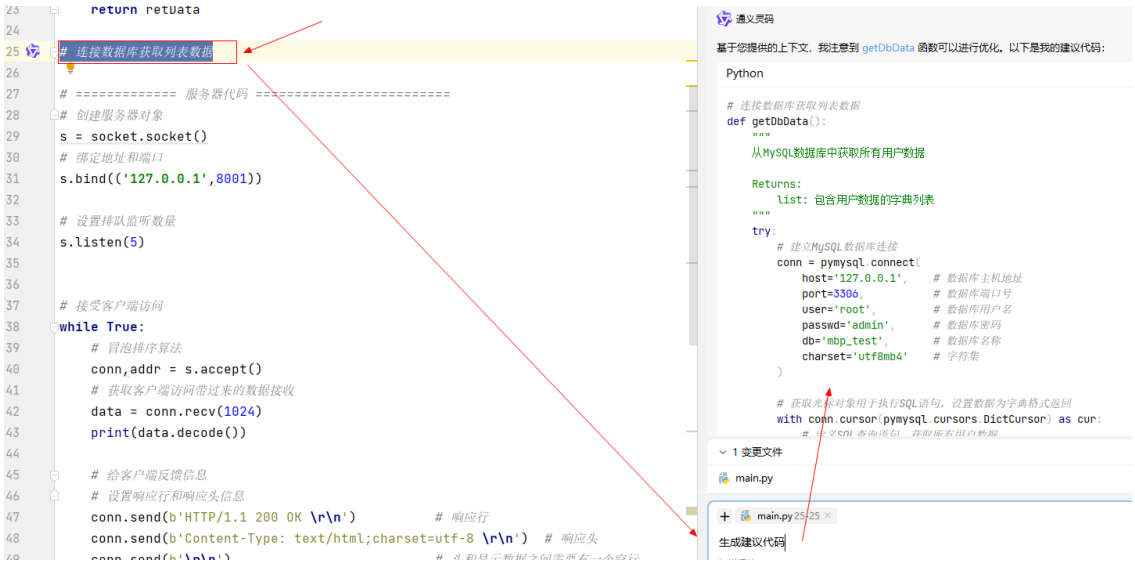
1. 直接问答
    1. 打开灵码窗口直接提问题即可
2. 根据文件中代码片段问答
    1. 选中需要执行操作的文件或文件中的代码片段
    2. 输入要执行的指令prompt

1.1.2 指令

- 生成建议代码
- 生成代码注释
- 代码优化
- 解释代码
- 修复代码
- 排查编译报错
- 联网查询 xxx 等

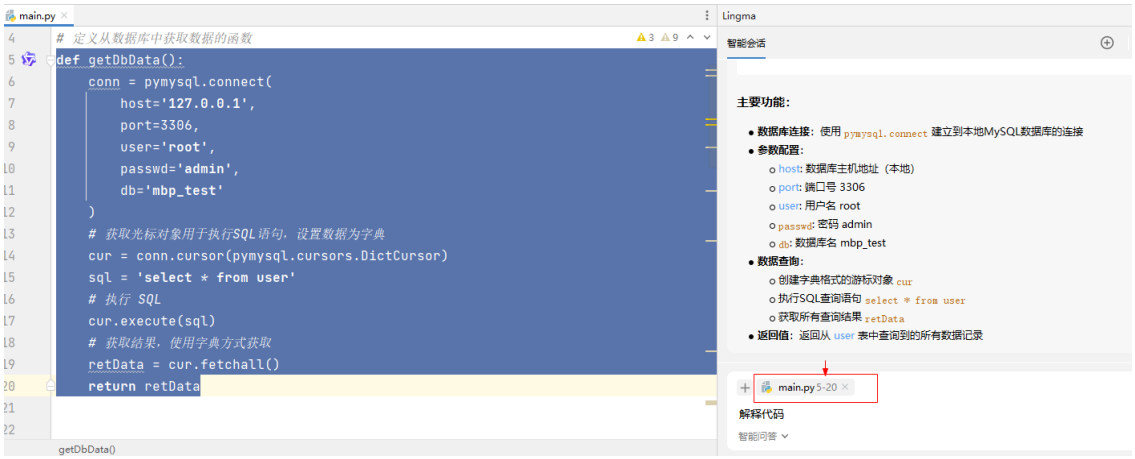
1.1.3 例子

1. 生成建议代码



2. 生成注释

3. 解释代码

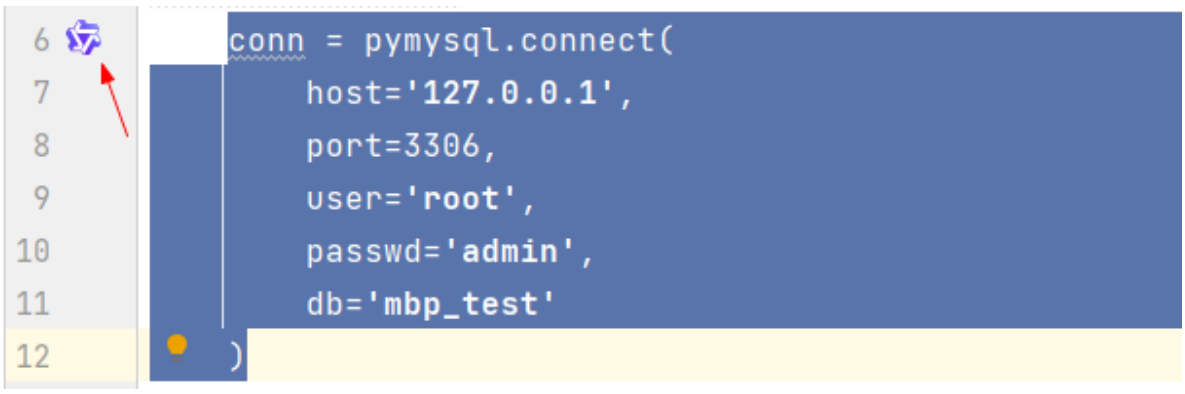


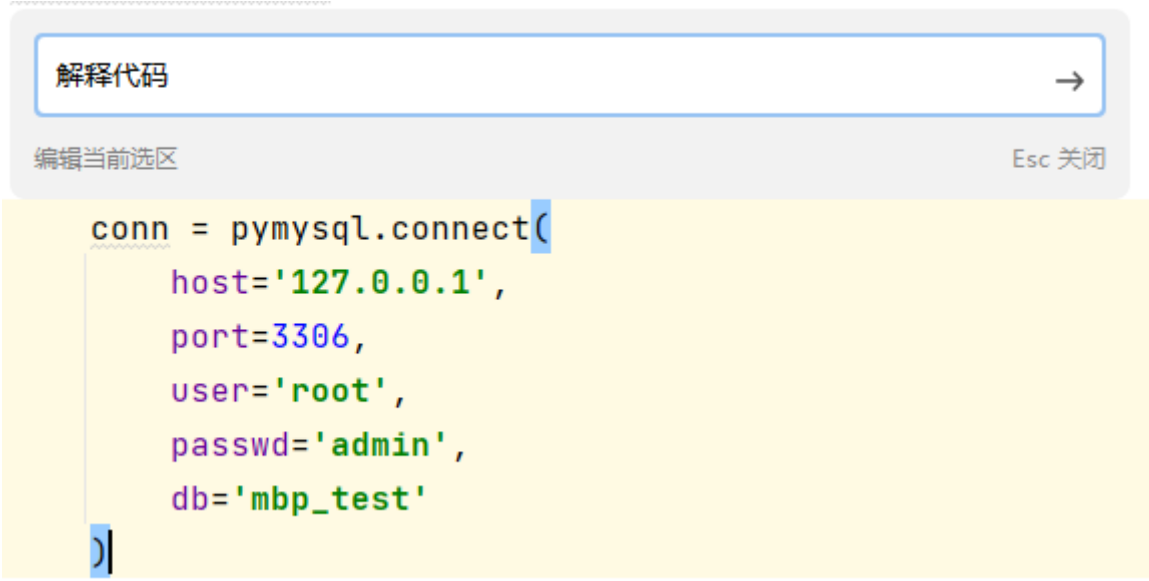
4. 优化代码

5. 联网查询python的基本语法

1.2 进阶智能问答

1. **打开指令输入框**
   - 选中需要 AI 处理的代码片段 --> 点击代码左边 lingma 图表





2. **输入指令**
    - 优化代码
    - 生成注释
    - 生成单元测试（选用的是函数代码来生成单元测试）
    - 解释代码

1.3 智能体

智能体模式具备自主决策、环境感知、工具使用等能力，可以根据开发者的编码诉求，使用工程检索、文件编辑、终端等工具，端到端地完成编码任务。同时，支持开发者配置 MCP 工具，拓宽了 AI 编码助手能力边界，更加贴合开发者工作流程。

智能体和智能问答的区别在于智能体能够直接调用工具来操作代码，而智能问答只是和AI进行对话

1.3.1 步骤

1. **直接问答**
    1. 选择 **智能体** 模式
    2. 对话窗口直接问答
2. **代码片段问答**
    1. 选择智能体模式
    2. 选择代码文件或代码片段
    3. 输入指令
    4. 选择是否接受智能体的操作

1.3.2 指令

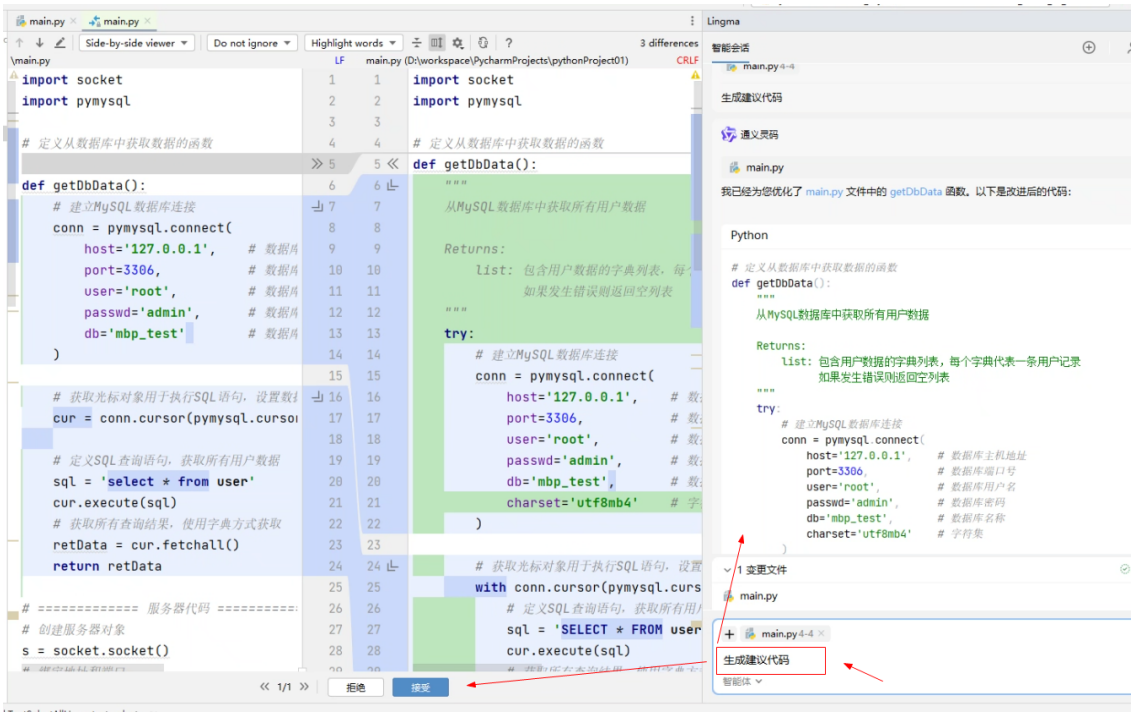
- 生成建议代码
- 生成代码注释
- 代码优化
- 解释代码
- 修复代码
- 排查编译报错

- 联网查询 xxx 等

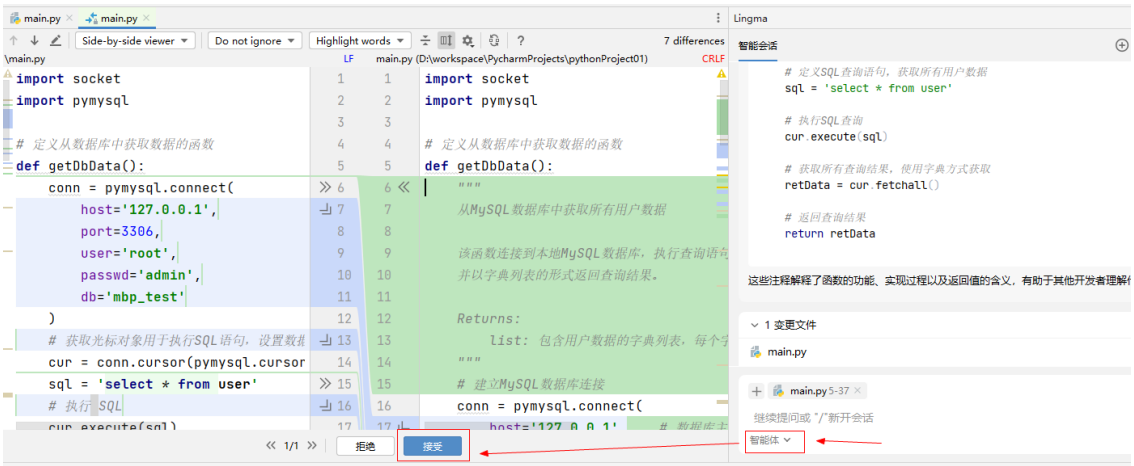
1.3.3 例子

- 生成建议代码

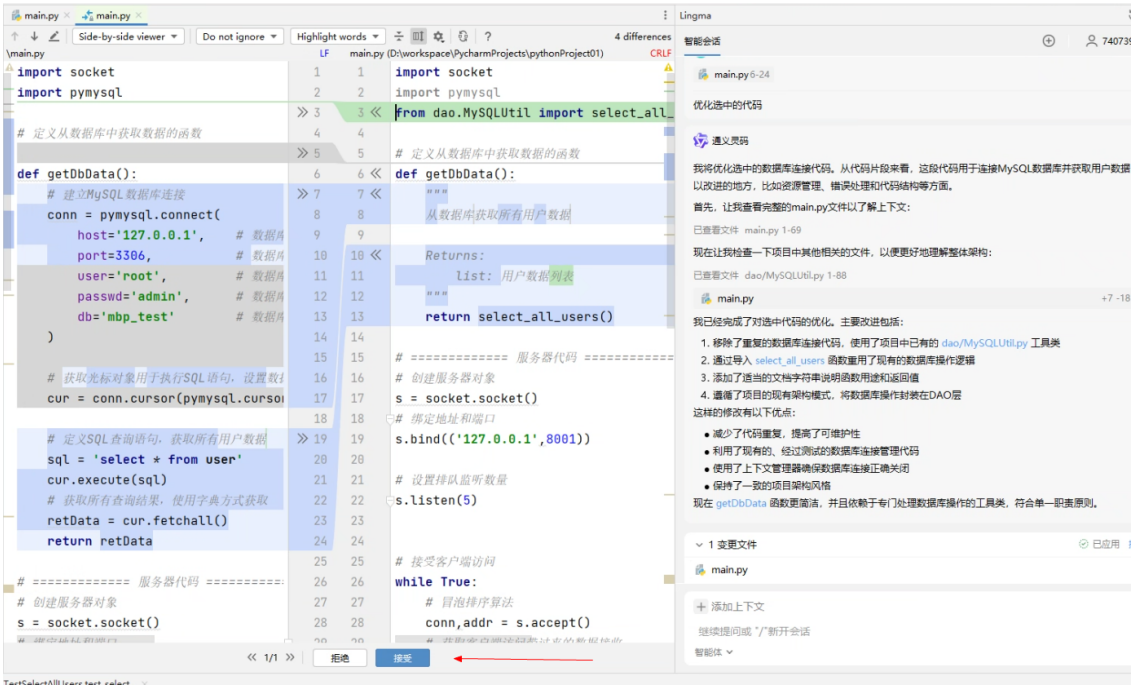
  选中注释：# 定义从数据库中获取数据的函数 --> 输入指令



- 生成注释

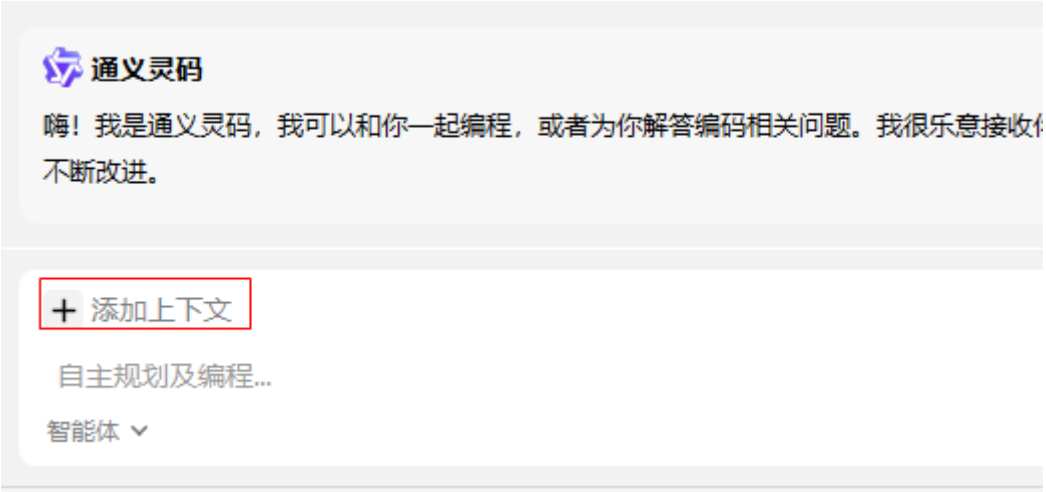


- 代码优化



1.4 上下文

智能会话中支持丰富的上下文，如代码文件、目录、图片、gitCommit 等，并提供上下文和用户输入提示词自由组装能力，开发者可以自由组合进行诉求描述。



1.4.1 添加上下文

在智能会话中，可以通过 3 种方式添加上下文

1. 输入框中单击添加上下文按钮，即可触发上下文选择窗口；
2. 输入框输入 `#` 或 `@` ，即可触发上下文选择窗口，继续输入时可进行文件检索；

> 说明
>
> - Lingma IDE和 JetBrains IDE 插件中使用`@`触发上下文选择窗口。
> - Visual Studio Code中使用`#`触发上下文选择窗口。

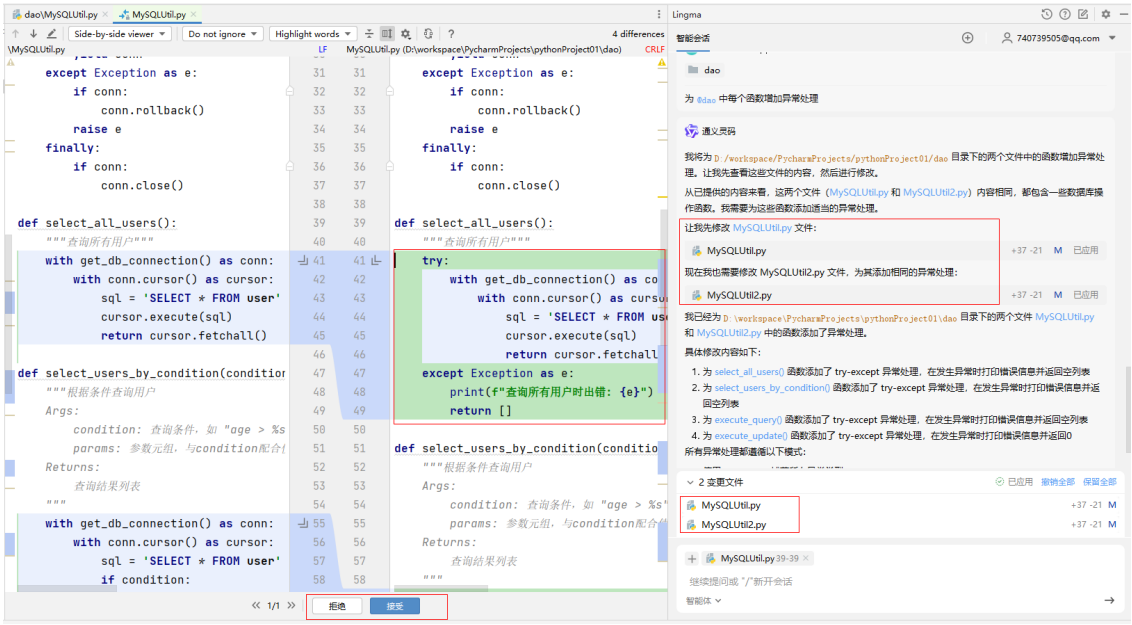
3. 对于代码文件、图片，可拖拽或复制粘贴，将对应内容添加为上下文（仅 JetBrains IDEs）
4. 可以将添加的上下文（标签）拖拽到输入的 指令 描述当中，方便进行输入组合

1.4.2 使用

上下文的使用主要是自行指定针对那些文件或代码来执行特定的指令，常用的有多文件同时操作，@file`、`@folder`、`@gitCommit`、`@rule，其中用的比较多的是 @folder 和 @rule

例子：需改指定目录下所有的代码文件

- 为每个方法增加异常处理



1.5 记忆

通义灵码提供了长期记忆能力，在开发者与通义灵码的对话过程，会逐渐形成针对开发者个人、工程、问题等相关的丰富记忆，并自动进行相关的记忆整理和更新。记忆能力可以帮助通义灵码更好地和开发者互动，随着时间流逝，也能够让通义灵码越来越懂开发者。

1.5.1 主动记忆

当需要通义灵码记住某些内容时，可以在智能会话窗口中，切换到智能体模式，直接输入通义灵码需要记忆的内容，发送后通义灵码即可创建相关的记忆。

1.5.2 自动记忆

通常情况下，通义灵码会主动记忆开发者与通义灵码的对话信息，逐步形成对开发者个人编码习惯、工程相关的记忆，开发者无需主动触发。

1.5.3 查看记忆

在智能会话窗口中，可通过单击用户名-个人设置-记忆管理，查看已经形成的记忆内容、作用范围等。同时，对于不需要的记忆，可以进行删除。

1.6 规则设置

通义灵码支持项目专属规则（Project Rules）的设定， 这些规则存储在 `.lingma/rules` 目录下，仅对当前工程生效。通过设定项目专属规则，可以帮助模型更精准地理解并适应您的编码偏好，例如理解项目框架和代码风格等。

1.6.1 规则限制

- 单个规则文件最大限制为 10000 字符，超过部分将自动截断。
- 规则文件请使用自然语言描述，不支持图片或链接的解析。

1.6.2 规则类型

根据引入和触发方式的不同，将规则分为以下4个类型，以灵活适配各类业务场景。

类型	触发方式	适用场景
手动引入（Manual）	在智能会话或行间会话中通过 `#rule` 手动引入才生效。	按需执行一次性或特定的工作流、自定义提示词等。
模型决策（Model Decision）	在智能会话中使用智能体模式时，或在智能问答开启工具使用后，模型将根据用户定义规则的描述内容，自主决策是否应用该规则。	由模型自主决策，仅在特定场景下生效的规则，例如生成单测时生效，或生成注释时生效等。
始终生效（Always）	在智能会话与行间会话中的所有请求中均会生效。	设定项目级的通用规范，如编码风格、偏好格式、默认的回答角色等
指定文件生效（Specific Files）	在智能会话和行间会话中，根据用户指定的文件匹配模式（如 `.js`、`src/**/.ts`），此规则将应用于所有符合该模式的文件。	可以根据通配符精准匹配生效的文件范围，为特定语言或目录创建专属规则。例如，针对某类语言生效的规则，或针对某个文件夹生效的校验规则。

1.6.3 规则创建

点击 灵码窗口 右上角个人头像 --> 选择个人设置 --> 规则 --> 点击 * 条记录 -> 添加规则 --> 输入规则文件名并选择类型

1.6.4 规则配置

规则文件编写需遵循以下做法：

- **保持简洁明确：**保持规则简洁、明确、具体。过长或含糊不清的规则可能会让通义灵码感到困惑。
- **结构化表达：**使用项目符号、编号列表和 Markdown 格式来格式化您的规则。与长段落相比，这些格式更易于通义灵码理解。
- **提供示例**：在规则中提供“好的代码示例”（Do），这能极大地帮助模型理解您的意图。
- **迭代优化**：创建规则后，通过实际的代码生成和问答来测试其效果，并根据模型的反馈不断迭代和优化您的规则描述。

python规则模板：

```
## 技术栈规范
### 框架与工具
1. 核心框架：
2. 依赖管理：
3. 测试框架：
4. 数据库：

## 代码结构规范
### 项目目录结构
### 代码风格
### 模块结构
### 注释规范（可读性规则）
### 测试类规范

## 数据库规范
### 数据库信息
### 模型设计
### 查询规范

## API开发规范
### 类存放规范
### 逻辑规范（可靠性规则）
### 安全规范（防御性规则）
```

案例：

my_crud_rule.md